

11. Generative models

16.12.2025

ETH zürich

D MATH

In today's class...

11.1 Generative modeling

- Introduction
- Applications in finance

11.2 Restricted Boltzmann Machines (RBMs)

- Definition
- Training

11.3 Variational Autoencoder (VAEs)

- Introduction
- Evidence Lower Bound
- Reparametrization trick

11.4 Normalizing Flows (NFs)

- Introduction
- Representational power
- RealNVP
- Autoregressive flows

11.1 Generative modeling

Generative modeling: given samples from a data distribution p_{data} on some measurable space $\mathcal{X}$ (e.g. $\mathbb{R}^d$, images, text, time-series, graphs), we want to learn a parametric distribution p_{θ} such that

$$p_{\theta} \approx p_{\text{data}}.$$

Learning is done by minimizing some notion of distance between the two distributions:

$$\theta^* = \arg \min_{\theta \in \Theta} d(p_{\text{data}}, p_{\theta})$$

where d is commonly:

- the Kullback-Leibler divergence (or other divergence),
- a metric on the space of distributions (e.g. Wasserstein distance)
- a score function.

11.1 Generative modeling

Eventually, we want to use generative models to perform the following operations:

- **sampling**, i.e. generate synthetic data $x \sim p_\theta$,
- **density evaluation**, i.e. compute $p_\theta(x)$,
- **representation learning**, i.e. find the posterior distribution $p_\theta(z|x)$, where z is a high-level latent variable (e.g. dog breed) and x is a data point (e.g. a dog image).

Due to computational trade-offs, at the moment there is no single generative model that can perform efficiently all three operations.

For each generative model in this class, we will answer the following questions:

- What is the parametric family $\{p_\theta, \theta \in \Theta\}$?
- What loss function is being minimized?
- What is the optimization algorithm?

11.1 Generative modeling: applications in finance

Generative models are used in several financial applications:

- ***Generative Data Augmentation (GDA)***: deep hedging models, for instance, need to be trained on price time series, but historical data is often scarce. One solution is to train a generative model on historical data and train the deep hedger on a dataset augmented with generated data.
- ***Risk measure estimation***: estimate risk measures on generated market scenarios from a generative model trained to replicate tail risk events, as in the TailGAN model [Cont et al., 2025].
- ***Model building***: model stochastic processes with structural constraints, as in neural SDEs [Kidger et al., 2021] and diffusion models with factor structure [Chen et al., 2025].
- ***Privacy-preserving data generation***: generate and share fully anonymized synthetic datasets reproducing sensitive data (e.g health insurance or proprietary information).
- ***Anomaly detection***: online outlier detection via score/likelihood evaluation.
- And many more...

Restricted Boltzmann Machines

11.2 Restricted Boltzmann Machines

A brief history:

- [Hopfield, 1982]: first energy-based recurrent neural networks.
- [Ackley et al., 1985]: Boltzmann machines (BMs).
- 1980s-1990s: several papers expose the computational intractability of BMs
- 2002-2006: in a series of papers Hinton et al. introduce Restricted Boltzmann Machines.

The model is famous because:

- It was the first successful energy-based model.
- It was the main component of the first deep learning model (the Deep Belief Network).
- It was used for pre-training of deep FNNs (2006–2010) (this practice is now outdated).

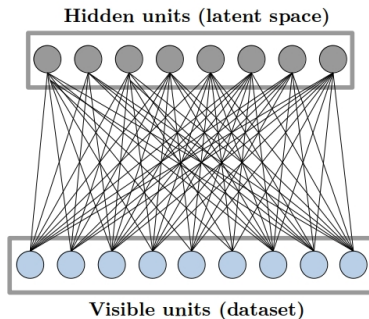
Nowadays:

- RBMs are considered too slow (due to MCMC, as we will see) and other models have gained in popularity (e.g. VAEs, GANs, Normalizing Flows, diffusion models).
- But many ML researchers expect energy-based models to play an important role in AGI (see so-called “world models” [Dawid and LeCun, 2024]).

Additional resources: [Fischer and Igel, 2012, Hinton, 2012]

11.2 RBMs: definition

A **Restricted Boltzmann Machine (RBM)** is a probabilistic graphical model that can be used to learn data distributions in an unsupervised way.



- Each vertex/node/unit represents a random variable (either *visible* or *hidden*),
- Edges represent direct statistical dependencies
- Bipartite structure¹ $\implies$ visible units are conditionally independent given the hidden units,
- Think of the hidden units as stochastic latent factors (non-linear factor analysis or PCA).

¹The term *restricted* is due to this bipartite structure. Boltzmann Machines, on the other hand, have full connectivity.

11.2 RBMs: definition

For binary random variables $V = (V_1, \dots, V_n)$ and $H = (H_1, \dots, H_m)$, the RBM is given by the following *Gibbs measure*:

$$p(v, h) = \frac{1}{Z} e^{-E(v, h)}, \quad v \in \{0, 1\}^n, h \in \{0, 1\}^m,$$

where

- $E(v, h)$ is the *energy function* with parameters $\theta = (W, b, c)$:

$$E(v, h) = - \underbrace{\sum_{i=1}^n v_i b_i}_{\text{visible bias}} - \underbrace{\sum_{i=1}^m h_i c_i}_{\text{hidden bias}} - \underbrace{\sum_{i=1}^n \sum_{j=1}^m W_{ij} v_i h_j}_{\text{interaction term}}$$

- Z is the *partition function* (just a normalization constant):

$$Z = \sum_{v \in \{0, 1\}^n} \sum_{h \in \{0, 1\}^m} e^{-E(v, h)}$$

11.2 RBMs: training

RBMs are trained by likelihood maximization via stochastic gradient ascent on the model parameters $\theta = (W, b, c)$:

$$\theta_{k+1} = \theta_k + \eta \nabla_{\theta} \log p(v),$$

where v is a datapoint in the training set and $\nabla_{\theta} \log p(v)$ is given by:

$$\begin{aligned}\nabla_{\theta} \log p(v) &= \nabla_{\theta} \log \sum_h p(v, h) \\&= \nabla_{\theta} \log \left(\frac{1}{Z} \sum_h e^{-E(v, h)} \right) && \text{(Use definition of } p(v, h) \text{)} \\&= \nabla_{\theta} \left(\log \sum_h e^{-E(v, h)} - \log \sum_{v', h} e^{-E(v', h)} \right) && \text{(Recall: } Z = \sum_{v, h} e^{-E(v, h)} \text{)} \\&= - \frac{1}{\sum_h e^{-E(v, h)}} \sum_h e^{-E(v, h)} \nabla_{\theta} E(v, h) + \frac{1}{\sum_{v', h} e^{-E(v', h)}} \sum_{v', h} e^{-E(v', h)} \nabla_{\theta} E(v', h) \\&= - \sum_h p(h|v) \nabla_{\theta} E(v, h) + \sum_{v', h} p(v', h) \nabla_{\theta} E(v', h)\end{aligned}$$

11.2 RBMs: training

Therefore at each gradient ascent step, the gradient is given by

$$\nabla_{\theta} \log p(v) = - \sum_h p(h|v) \nabla_{\theta} E(v, h) + \sum_{v', h} p(v', h) \nabla_{\theta} E(v', h)$$

- The distribution $p(h|v)$ is easy to compute (check it!):

$$\mathbb{P}(H_j = 1|v) = \sigma \left(\sum_{i=1}^n W_{ji} v_i + c_j \right), \quad \forall j = 1, \dots, m,$$

where σ is the sigmoid function and v is a training sample.

- Exact evaluation of $p(v, h)$, on the other hand, is intractable, because computing Z requires $2^{\min(n, m)}$ additions at each gradient step $\implies$ exact training is prohibitive!
- Instead, we approximate samples from $p(v, h)$ using Gibbs sampling, a Markov Chain Monte Carlo (MCMC) technique, in which we generate samples from a Markov Chain with guaranteed stationary distribution equal to $p(v, h)$.

11.2 RBMs: training

In particular, we use *blocked Gibbs sampling*.

Data: RBM with parameters (W, b, c) ; T = number of Gibbs steps.

Result: Sample (V, H) from RBM distribution.

Sample V from the uniform distribution on $\{0, 1\}^n$.

for $k = 1$ **to** T **do**

 Sample H given V using:

$$\mathbb{P}(H_j = 1 \mid V) = \sigma \left(\sum_{i=1}^n W_{ij} V_i + c_j \right), \quad \forall j = 1, \dots, m$$

 Sample V given H using:

$$\mathbb{P}(V_i = 1 \mid H) = \sigma \left(\sum_{j=1}^m W_{ij} H_j + b_i \right), \quad \forall i = 1, \dots, n$$

end

return (V, H) .

11.2 RBMs: training

In practice the number of Gibbs steps T can be reduced using algorithms exist:

- **Contrastive Divergence**: blocked Gibbs sampling for only a few steps [Hinton, 2002].
- **Persistent Contrastive Divergence**: decrease T by smart initialization [Tieleman, 2008].
- **Parallel Tempering**: reduce T using annealed sampling [Salakhutdinov, 2009].

There are also other (not likelihood-based) training algorithms:

- Pseudo-likelihood,
- ratio-matching,
- denoising score-matching, ...

RBMs can be used to model images, audio tracks, time-series, etc

- Binary RBMs: sample $\mathbb{P}(V_i | H)$ instead of V_i for data in $[0, 1]^n$ (e.g. B&W images)
- Gaussian-Bernoulli RBMs
- Gaussian-Gaussian RBMs [Frey, 1996]
- Exponential Family RBMs [Welling et al., 2004]
- Spike-and-Slab (ssRBMs) [Courville et al., 2011]

Variational Auto-Encoders

11.3 Variational Autoencoders

VAEs were introduced independently in **two separate papers**:

- [Kingma and Welling, 2014]: neural-based autoencoding with reparameterization trick.
- [Rezende et al., 2014]: deep latent Gaussian model with stochastic back-propagation.

Most **successful variants** to date:

- β -VAE [Higgins et al., 2017],
- Wasserstein AutoEncoders (WAEs) [Tolstikhin et al., 2017],
- VAEs with Normalizing Flows (NF-VAE)
[Rezende and Mohamed, 2015, Kingma et al., 2016]

Additional resources: [Kingma et al., 2019]

11.3 VAEs: introduction

We want to approximate a data distribution $p_{\text{data}}(x)$ using a probabilistic model involving latent variables:

$$p_{\theta}(x) = \int p_{\theta}(x | z)p(z)dz$$

where $\theta \in \Theta$ is a vector of parameters and $p(z)$ is a fixed data **prior** for the *latent variable* z .

First of all, **why should we introduce the latent variables z ?**

- even a simple $p_{\theta}(x | z)$ can generate a complex $p_{\theta}(x)$,
- z captures low-dimensional structural features of the data,
- z can be used to perform dimensionality reduction,
- $p(z)$ enforces smoothness and regularity (helps with sampling and interpolation).

11.3 VAEs: introduction

One approach is to minimize the Kullback-Leibler (KL) divergence:

$$\mathcal{D}_{KL}(p_{\text{data}}(x) \parallel p_{\theta}(x)) := \int p_{\text{data}}(x) \log \left(\frac{p_{\text{data}}(x)}{p_{\theta}(x)} \right) dx$$

- $\mathcal{D}_{KL}$ is a statistical distance between probability distributions,
- $\mathcal{D}_{KL}(p \parallel q) \geq 0$ for all p, q ,
- $\mathcal{D}_{KL}(p \parallel q) = 0$ if and only if $p = q$,
- but it's not symmetric ($\mathcal{D}_{KL}(p \parallel q) \neq \mathcal{D}_{KL}(q \parallel p)$) and therefore not a metric,
- $\mathcal{D}_{KL}(p \parallel q)$ is well-defined only if $p \ll q$,
- minimizing the empirical KL divergence is equivalent to maximizing the likelihood:

$$\begin{aligned} \inf_{\theta \in \Theta} \mathcal{D}_{KL}(p_{\text{data}}(x) \parallel p_{\theta}(x)) &= \inf_{\theta \in \Theta} \left\{ - \int p_{\text{data}}(x) \log(p_{\theta}(x)) dx + \text{constant} \right\} \\ &= \sup_{\theta \in \Theta} \int p_{\text{data}}(x) \log(p_{\theta}(x)) dx \\ &\approx \sup_{\theta \in \Theta} \frac{1}{m} \sum_{i=1}^m \log(p_{\theta}(x_i)) \end{aligned}$$

11.3 VAEs: Evidence Lower Bound

- We could use Monte Carlo estimation on a sample $(z_1, \dots, z_k) \stackrel{\text{iid}}{\sim} p(z)$ and compute:

$$p_{\theta}(x) = \int p_{\theta}(x | z)p(z)dz \approx \frac{1}{k} \sum_{j=1}^k p_{\theta}(x | z_j)$$

- But in practice the estimator suffers from very high variance and instability.

Main idea of VAEs: introduce an approximate variational posterior $q_{\phi}(z | x) \approx p_{\theta}(z | x)$ (with parameters $\phi \in \Phi$) and maximize a lower bound on $\log p_{\theta}(x)$.

$$\begin{aligned} \log p_{\theta}(x) &= \log \left(\int \frac{q_{\phi}(z | x)}{q_{\phi}(z | x)} p_{\theta}(x, z) dz \right) \\ &\geq \int q_{\phi}(z | x) \log \left(\frac{p_{\theta}(x, z)}{q_{\phi}(z | x)} \right) dz && \text{(Jensen's inequality)} \\ &= \mathbb{E}_{q_{\phi}(\cdot | x)} \left[\log \left(\frac{p_{\theta}(x, z)}{q_{\phi}(z | x)} \right) \right] \\ &=: \text{ELBO}(\theta, \phi | x) && \text{(ELBO = Evidence Lower BOund)} \end{aligned}$$

11.3 VAEs: Evidence Lower Bound

Therefore, we maximize the empirical ELBO as a proxy for likelihood maximization in (θ, ϕ) :

$$(\theta^*, \phi^*) = \arg \max_{(\theta, \phi) \in \Theta \times \Phi} \frac{1}{m} \sum_{i=1}^m \text{ELBO}(\theta, \phi \mid x_i).$$

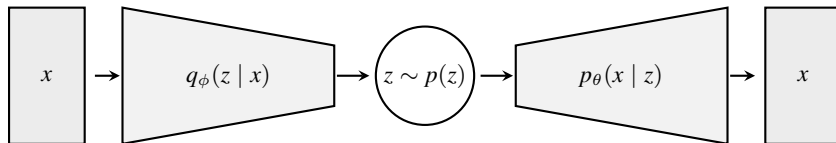
In practice, we maximize the following equivalent expression of ELBO:

$$\begin{aligned} \text{ELBO}(\theta, \phi \mid x) &= \int q_{\phi}(z \mid x) \log \left(\frac{p_{\theta}(x, z)}{q_{\phi}(z \mid x)} \right) dz \\ &= \int q_{\phi}(z \mid x) \left(\log p_{\theta}(x \mid z) - \log \left(\frac{q_{\phi}(z \mid x)}{p(z)} \right) \right) dz \\ &= \underbrace{\int q_{\phi}(z \mid x) \log p_{\theta}(x \mid z) dz}_{\text{Reconstruction term}} - \underbrace{\mathcal{D}_{KL}(q_{\phi}(z \mid x) \parallel p(z))}_{\text{KL "regularization" term}} \end{aligned}$$

- The **reconstruction term** pushes $p_{\theta}(x \mid z)$ to fit the input data.
- The **KL “regularization” term** matches $q_{\phi}(z \mid x)$ to the prior $p(z)$, to ensure:
 - meaningful latent space structure,
 - accurate data generation after training ($z \sim p(z) \Rightarrow x \sim p_{\theta}(x \mid z)$).

11.3 VAEs as autoencoders

Think of a VAE as a *probabilistic* auto-encoder (a stochastic neural network):



- $q_\phi(z | x)$ **encodes** input data x into latent representations z ,
- $p_\theta(z | x)$ **decodes** latent representations z into input data x .

For numerical, continuous input data, it is very common to assume:

$$q_\phi(z | x) \sim \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x)))$$

$$p_\theta(x | z) \sim \mathcal{N}(\mu_\theta(z), \sigma^2 I)$$

$$p(z) \sim \mathcal{N}(0, I)$$

where $\mu_\phi, \sigma_\phi^2, \mu_\theta$ are feedforward neural networks and $\sigma > 0$ is a constant.

11.3 VAEs: KL "regularization" term

Let's see how we can compute the KL "regularization" term.

The KL divergence of two d -dimensional Gaussian distributions $q \sim \mathcal{N}(\mu_q, \Sigma_q)$, $p \sim \mathcal{N}(\mu_p, \Sigma_p)$, admits a closed-form formula:

$$\mathcal{D}_{KL}(q \parallel p) = \frac{1}{2} \left(\text{Tr}(\Sigma_q \Sigma_p^{-1}) + (\mu_p - \mu_q)^T \Sigma_p^{-1} (\mu_p - \mu_q) - d + \log \left(\frac{\det(\Sigma_p)}{\det(\Sigma_q)} \right) \right).$$

Under our assumptions

$$q_\phi(z \mid x) \sim \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x))) \quad \text{and} \quad p(z) \sim \mathcal{N}(0, I),$$

we obtain:

$$\mathcal{D}_{KL}(q_\phi(z \mid x) \parallel p(z)) = \frac{1}{2} \sum_{i=1}^d (\sigma_{\phi,i}^2(x) + \mu_{\phi,i}^2(x) - 1 - \log(\sigma_{\phi,i}^2(x))),$$

where $\sigma_{\phi,i}(x)$ and $\mu_{\phi,i}(x)$ are the i -th output coordinates of the networks $\sigma_\phi(x)$ and $\mu_\phi(x)$.

11.3 VAEs: the reparameterization trick

Computing the reconstruction term is slightly more difficult.

We estimate the reconstruction term using Monte Carlo on an i.i.d. sample $(z_1, \dots, z_k) \sim q_\phi(\cdot | x)$:

$$\begin{aligned}\int q_\phi(z | x) \log p_\theta(x | z) dz &\approx \frac{1}{k} \sum_{j=1}^k \log p_\theta(x | z_j) && \text{(Monte Carlo)} \\ &= -\frac{1}{k} \sum_{j=1}^k \frac{1}{2\sigma^2} \|x - \mu_\theta(z_j)\|^2 + \text{constant} && (p_\theta(x | z) \sim \mathcal{N}(\mu_\theta(z), \sigma^2 I))\end{aligned}$$

Problem: The sampling distribution $q_\phi(\cdot | x)$ depends on ϕ , but we cannot back-propagate the gradient for the parameters ϕ through the sampling operation!

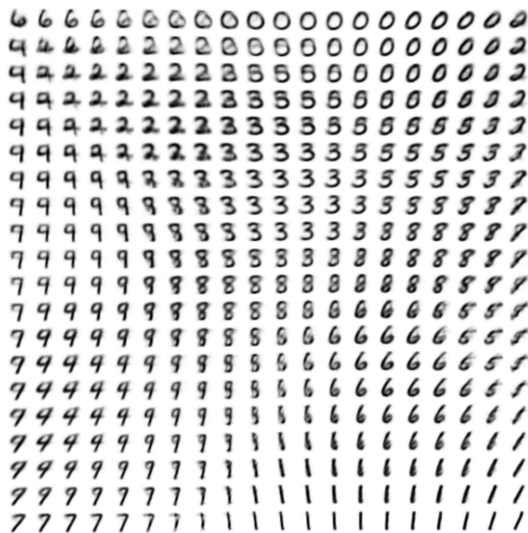
Solution: Rewrite each sample $z \sim q_\phi(\cdot | x)$ as

$$z = \mu_\phi(x) + \text{diag}(\sigma_\phi^2(x)) \cdot \varepsilon,$$

where $\varepsilon \sim \mathcal{N}(0, I)$ does not depend on ϕ .

This is known as the “**reparameterization trick**” and it allows the end-to-end training of both encoder q_ϕ and decoder p_θ via backpropagation.

11.3 VAEs: example of meaningful latent embedding



Two-dimensional latent space of VAE trained on the MNIST dataset [[Kingma and Welling, 2014](#), Figure 4(b)].

Normalizing Flows

11.4 Normalizing Flows

Brief history:

- [Johnson, 1966, Friedman, 1987]: whitening transformations (transform data into noise)
- [Tabak and Turner, 2013] introduce Normalizing Flows (transform noise into data)
- **RealNVP**: the first scalable and computationally efficient architecture [Dinh et al., 2014]
- Applications to variational inference [Rezende and Mohamed, 2015]
- **Glow**: top-performing image-generation model [Kingma and Dhariwal, 2018]

Additional resources: [Papamakarios et al., 2021, Kobyzev et al., 2020]

11.4 Normalizing Flows

Given a data distribution p_{data} , we want to represent our data $X \sim p_{\text{data}}$ as a transformation of some noise variable U :

$$X = T(U), \quad U \sim p_u,$$

where p_u is an easy-to-evaluate distribution (e.g. a multivariate standard normal), called the **base distribution** and T is a parametrized transformation to be learned.

Main idea of NFs: if the transformation T is invertible and both T and T^{-1} are differentiable (i.e. T is a **diffeomorphism**), then the change of variable formula yields:

$$\begin{aligned} p_x(x) &= p_u(u) |\det(J_T(u))|^{-1} & (u = T^{-1}(x)) \\ &= p_u(T^{-1}(x)) |\det(J_{T^{-1}}(x))| \end{aligned}$$

where J_T is Jacobian matrix of the transformation T .

This allows us to train the model by maximizing the exact likelihood:

$$\sum_{i=1}^m \log(p_x(x_i)) = \sum_{i=1}^m \log(p_u(T^{-1}(x_i)) - \log(|\det(J_{T^{-1}}(x_i))|)$$

11.4 Normalizing Flows

Typically T is the composition of K simple diffeomorphisms, called *flows*:

$$T = T_K \circ \cdots \circ T_1,$$

from which it follows immediately that

$$T^{-1} = T_1^{-1} \circ \cdots \circ T_K^{-1} \quad \text{and} \quad \log(|\det(J_{T^{-1}}(x))|) = \sum_{j=1}^K \log\left(|\det(J_{T_j^{-1}}(x))|\right).$$

This explain the name *normalizing flow*:

- *flow*, because every datapoint u traces a trajectory in $\mathbb{R}^d$ as it is being gradually transformed through the maps $T_1, \dots, T_K$,
- *normalizing*, because the inverse tranformation $T_1^{-1} \circ \cdots \circ T_K^{-1}$ maps the data distribution p_x into the base distribution p_u , which is commonly assumed to be normal.

11.4 NFs: representational power

Theorem 1 (UAP for NFs)

If X and U are two absolutely continuous random vectors in $\mathbb{R}^d$ and X has a strictly positive density, then there exists a diffeomorphism $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ such that $T(U)$ has the same distribution as X .

The proof follows from a generalization of the following well-known fact:

Recall: if X is a random variable with cdf F , then:

- $U = F(X) \sim \mathcal{U}[0, 1]$, and
- $X = F^{-1}(U)$ almost surely, where F^{-1} is the quantile function² of X .

Proof.

- It's enough to consider the case $U \sim \mathcal{U}[0, 1]^d$ (why?).
- Denote by p_x and p_u the densities of X and U respectively.
- The following autoregressive decomposition always holds:

$$p_x(x) = \prod_{i=1}^d p_x(x_i \mid x_1, \dots, x_{i-1}).$$

²The quantile function is the left-continuous generalized inverse of F . If F is strictly increasing (i.e. X admits a positive density almost everywhere), then F^{-1} is just the inverse of F .

11.4 NFs: representational power

Proof.

- Define the transformation $u = F(x)$, with $F : \mathbb{R}^d \rightarrow (0, 1)^d$ and i -th coordinate given by:

$$u_i = F_i(x_i \mid x_1, \dots, x_{i-1}) := \mathbb{P}(X_i \leq x_i \mid x_1, \dots, x_{i-1}) = \int_{-\infty}^{x_i} p_x(x'_i \mid x_1, \dots, x_{i-1}) dx'_i$$

Notice that $F_i(\cdot \mid x_1, \dots, x_{i-1})$ is almost everywhere differentiable and strictly increasing (its derivative is $p_x(\cdot \mid x_1, \dots, x_{i-1}) > 0$) and therefore invertible, $x_i = F_i^{-1}(u_i \mid x_1, \dots, x_{i-1})$.

- The Jacobian of F is lower-triangular because $\frac{\partial F_i}{\partial x_j} = 0$ for $i < j$, therefore its determinant is just the product of the diagonal entries:

$$\det(J_F(x)) = \prod_{i=1}^d \frac{\partial F_i}{\partial x_i} = \prod_{i=1}^d p_x(x_i \mid x_1, \dots, x_{i-1}) = p_x(x).$$

- Since $\det(J_F(x))$ is strictly positive, its inverse exists and is equal to the Jacobian of F^{-1} , therefore F is a diffeomorphism.

11.4 NFs: representational power

Proof.

- By the change of variable formula, notice that:

$$p_u(u) = p_x(x) |\det(J_F(x))|^{-1} = p_x(x) p_x(x)^{-1} = 1$$

therefore U is indeed uniformly distributed on $[0, 1]^d$.



The theory of optimal transport provides an alternative proof:

Theorem 2 ([Gangbo and McCann, 1996])

If μ and ν are two absolutely continuous, p -integrable measures on $\mathbb{R}^d$ (with $p > 1$), then there exists a unique solution to the following problem:

$$\inf_{T_{\#}\mu=\nu} \int_{\mathbb{R}^d} \|x - T(x)\|^p dx,$$

where T is almost surely a bijective map.

If the densities are smooth, then it can be shown that T is a diffeomorphism.

11.4 NFs: RealNVP

- RealNVP was the first Normalizing Flow capable of scalable training in high-dimensional settings ($d \approx 10,000$) [Dinh et al., 2014].
- They introduced the *affine coupling flow*, which is a simple but effective bijective transformation:

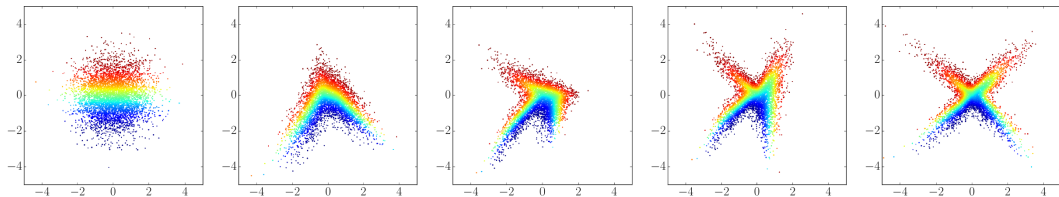
$$x'_j = \begin{cases} \sigma_j(x_{k+1}, \dots, x_d) x_j + \mu_j(x_{k+1}, \dots, x_d) & \text{if } j = 1, \dots, k \\ x_j & \text{if } j = k + 1, \dots, d \end{cases}$$

where $\sigma : \mathbb{R}^{d-k} \rightarrow \mathbb{R}_+^k$ and $\mu : \mathbb{R}^{d-k} \rightarrow \mathbb{R}^k$ are two feedforward neural networks.

- They alternated affine coupling flows with random *coordinate swaps*

$$(x'_1, \dots, x'_d) = (x_{\pi(1)}, \dots, x_{\pi(d)}),$$

where π is a random permutation chosen at initialization and kept fixed during training.



11.4 NFs: autoregressive flows

One can exploit the autoregressive decomposition in Theorem 1, by defining flows of the form:

$$x'_i = \tau(x_i; \theta_i), \quad \theta_i = c_i(x_1, \dots, x_{i-1}), \quad i = 1, \dots, d$$

where:

- $x \mapsto \tau(x; \theta)$ is an invertible function with parameters $\theta \in \Theta$, called the **transformer**³,
- $c : \mathbb{R}^d \rightarrow \Theta$ is an autoregressive function (i.e. $c_i(x) = c_i(x_1, \dots, x_{i-1})$), called the **conditioner**.

The transformation above can be inverted recursively:

$$x_i = \tau^{-1}(x'_i; \theta_i) \quad \theta_i = c_i(x_1, \dots, x_{i-1}), \quad i = 1, \dots, d.$$

It has a lower triangular Jacobian ($\tau(x_i; \theta_i)$ does not depend on x_j for all $j > i$), with easy-to-compute log-determinant:

$$\log |\det(J(x))| = \sum_{i=1}^d \log \left| \frac{\partial \tau}{\partial x_i}(x_i; c_i(x_1, \dots, x_{i-1})) \right|.$$

- Notice that the conditioner does **not** need to be invertible (e.g. can be a neural network).
- Transforming x into x' requires one forward pass, but the inverse requires d passes
⇒ sampling is fast, but density evaluation (required for training) is slow,
⇒ for efficient training, you can invert the flow direction.

³No relationship with transformer models, such as ChatGPT.

11.4 NFs: autoregressive flows

Common choices for autoregressive flows:

- **(Masked) affine transformers:**

$$\tau(x_i; \mu_i, \sigma_i) = \sigma_i x_i + \mu_i \quad (\mu_i, \sigma_i) = c_i(x_1, \dots, x_{i-1}), \text{ with } \sigma_i > 0,$$

where $c : \mathbb{R}^d \rightarrow \mathbb{R}^{2d}$ is an autoregressive feedforward neural network⁴.

- **Spline-based transformers:** $\tau(x_i; \theta_i)$ is a low-order monotonic spline (i.e. a piecewise polynomial function with low-degree between gridpoints $\{x_i^{(1)}, \dots, x_i^{(n)}\}$).
- **Neural transformers:** where the transformer is a shallow, invertible neural network, e.g.

$$\tau(x_i; w_i^{(2)}, b_i^{(2)}, w_i^{(1)}, b_i^{(1)}) = \sum_{j=1}^n w_{ij}^{(2)} \text{ReLU}(w_{ij}^{(1)} x_i + b_{ij}^{(1)}) + b_i^{(2)},$$

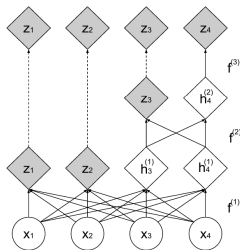
and $(w^{(2)}, b^{(2)}, w^{(1)}, b^{(1)}) = c(x)$ is an autoregressive *hypernetwork*⁵.

⁴Autoregressive feedforward networks are commonly implemented using the MADE technique [Mathieu, 2015], in which the hidden neurons of the networks are multiplied by a binary mask, chosen in such a way that the i -th output neuron depends only input coordinates $j = 1, \dots, i - 1$.

⁵A hypernetwork is a neural network whose outputs are the parameters of another network.

11.4 NFs: multi-scale architecture

- **Problem:** Brouwer's “invariance of domain” theorem implies that if T is a diffeomorphism between $\mathbb{R}^m$ and $\mathbb{R}^n$, then $m = n$
 $\implies p_u$ and p_x must be on a space of the same dimension.
- If the data space is very high-dimensional, then training can be slow.
- But we know empirically that many data types (e.g. images) have a low “effective” dimension⁶
 $\implies$ the base distribution p_u concentrates on a low-dimensional manifold.
- **Solution:** define a *multi-scale* architecture, in which only a fraction of noise variables are used as input in all flows (mapping the coordinates of the non-linear, latent manifold), while the majority of noise variables undergo only few flow transformations (capturing “background noise” in directions normal to the manifold).



⁶The effective dimension of the CIFAR-10 dataset (32x32 RGB images, i.e. $d = 3072$ in pixel space) has been estimated to be $d \approx 10$ [Gong et al., 2019].

References I

-  Ackley, D. H., Hinton, G. E., and Sejnowski, T. J. (1985).
A learning algorithm for boltzmann machines.
Cognitive science, 9(1):147–169.
-  Chen, M., Xu, R., Xu, Y., and Zhang, R. (2025).
Diffusion factor models: Generating high-dimensional returns with factor structure.
arXiv preprint arXiv:2504.06566.
-  Cont, R., Cucuringu, M., Xu, R., and Zhang, C. (2025).
Tail-gan: Learning to simulate tail risk scenarios.
Management Science.
-  Courville, A., Bergstra, J., and Bengio, Y. (2011).
A spike and slab restricted boltzmann machine.
In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 233–241. JMLR Workshop and Conference Proceedings.

References II

-  Dawid, A. and LeCun, Y. (2024).
Introduction to latent variable energy-based models: a path toward autonomous machine intelligence.
Journal of Statistical Mechanics: Theory and Experiment, 2024(10):104011.
-  Dinh, L., Krueger, D., and Bengio, Y. (2014).
Nice: Non-linear independent components estimation.
arXiv preprint arXiv:1410.8516.
-  Fischer, A. and Igel, C. (2012).
An introduction to restricted boltzmann machines.
In *Iberoamerican congress on pattern recognition*, pages 14–36. Springer.
-  Frey, B. J. (1996).
Continuous sigmoidal belief networks trained using slice sampling.
Advances in neural information processing systems, 9.

References III

-  Friedman, J. H. (1987).
Exploratory projection pursuit.
Journal of the American statistical association, 82(397):249–266.
-  Gangbo, W. and McCann, R. J. (1996).
The geometry of optimal transportation.
-  Gong, S., Boddeti, V. N., and Jain, A. K. (2019).
On the intrinsic dimensionality of image representations.
In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition,
pages 3987–3996.
-  Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., and Lerchner, A. (2017).
beta-vae: Learning basic visual concepts with a constrained variational framework.
In International conference on learning representations.

References IV

-  Hinton, G. E. (2002).
Training products of experts by minimizing contrastive divergence.
Neural computation, 14(8):1771–1800.
-  Hinton, G. E. (2012).
A practical guide to training restricted boltzmann machines.
In *Neural Networks: Tricks of the Trade: Second Edition*, pages 599–619. Springer.
-  Hopfield, J. J. (1982).
Neural networks and physical systems with emergent collective computational abilities.
Proceedings of the national academy of sciences, 79(8):2554–2558.
-  Johnson, R. M. (1966).
The minimal transformation to orthonormality.
Psychometrika, 31(1):61–66.
-  Kidger, P., Foster, J., Li, X., and Lyons, T. J. (2021).
Neural sdes as infinite-dimensional gans.
In *International conference on machine learning*, pages 5453–5463. PMLR.

References V

-  Kingma, D. P. and Dhariwal, P. (2018).
Glow: Generative flow with invertible 1x1 convolutions.
Advances in neural information processing systems, 31.
-  Kingma, D. P., Salimans, T., Jozefowicz, R., Chen, X., Sutskever, I., and Welling, M. (2016).
Improved variational inference with inverse autoregressive flow.
Advances in neural information processing systems, 29.
-  Kingma, D. P. and Welling, M. (2014).
Auto-encoding variational bayes.
In International Conference on Learning Representations. PMLR.
-  Kingma, D. P., Welling, M., et al. (2019).
An introduction to variational autoencoders.
Foundations and Trends® in Machine Learning, 12(4):307–392.
-  Kobzyev, I., Prince, S. J., and Brubaker, M. A. (2020).
Normalizing flows: An introduction and review of current methods.
IEEE transactions on pattern analysis and machine intelligence, 43(11):3964–3979.

References VI

-  Mathieu, M. (2015).
Masked autoencoder for distribution estimation.
-  Papamakarios, G., Nalisnick, E., Rezende, D. J., Mohamed, S., and Lakshminarayanan, B. (2021).
Normalizing flows for probabilistic modeling and inference.
Journal of Machine Learning Research, 22(57):1–64.
-  Rezende, D. and Mohamed, S. (2015).
Variational inference with normalizing flows.
In *International conference on machine learning*, pages 1530–1538. PMLR.
-  Rezende, D. J., Mohamed, S., and Wierstra, D. (2014).
Stochastic backpropagation and approximate inference in deep generative models.
In *International conference on machine learning*, pages 1278–1286. PMLR.
-  Salakhutdinov, R. R. (2009).
Learning in markov random fields using tempered transitions.
Advances in neural information processing systems, 22.

References VII

-  Tabak, E. G. and Turner, C. V. (2013).
A family of nonparametric density estimation algorithms.
Communications on Pure and Applied Mathematics, 66(2):145–164.
-  Tieleman, T. (2008).
Training restricted boltzmann machines using approximations to the likelihood gradient.
In *Proceedings of the 25th international conference on Machine learning*, pages 1064–1071.
-  Tolstikhin, I., Bousquet, O., Gelly, S., and Schoelkopf, B. (2017).
Wasserstein auto-encoders.
arXiv preprint arXiv:1711.01558.
-  Welling, M., Rosen-Zvi, M., and Hinton, G. E. (2004).
Exponential family harmoniums with an application to information retrieval.
Advances in neural information processing systems, 17.